

SUMMARY

This video explains the methodologies for constructing Minimal Deterministic Finite Automata (DFA) and analyzing Compound (Product) Automata. The focus is on systematic state-count shortcuts (AD Rules) and efficient logical composition techniques relevant to competitive examinations such as GATE, ISRO, and BARC.

Key Takeaways:

- State-count formulas (Minimal Length + 1, +2, or +3) enable direct computation of minimal DFA states.
- Compound automata use product construction ($m \times n$ states).
- Modulo-based languages require exactly n states for divisor n .
- Conceptual clarity eliminates the need for full diagram construction in many problems.

PART I: MINIMAL DFA STATE CALCULATION (AD RULES)

Languages of the form $a^m b^n$ are classified using binary-style logic (0 = no lower bound, 1 = lower bound $\geq k$).

1. Language Categorization and State Formulas

Category	Constraint Logic	Example Language	Minimal States Formula
Type A	$m \geq 0, n \geq 0$	$a^m b^n$	Alphabet Length + 1
Type B	$m \geq 0, n \geq k$	$a^m b^n$	Minimal Length + 2
Type C	$m \geq k, n \geq 0$	$a^m b^n$	Minimal Length + 3
Type D	$m \geq k, n \geq j$	$a^m b^n$	Minimal Length + 2

2. Detailed Category Analysis

Type A: All Variables ≥ 0

Most flexible structure.

Example:

$L = a^m b^n c^p$ where $m, n, p \geq 0$

Alphabet size = 3
States = $3 + 1 = 4$

General Case:

Alphabet a through z (26 symbols)
States = $26 + 1 = 27$

Type B: Second Variable Incrementing

Example:

$a^m b^n$, $m \geq 0$, $n \geq 1$

Minimal string = "b"
Minimal length = 1

States = $1 + 2 = 3$

Example:

$a^m b^n$, $m \geq 0$, $n \geq 2025$

States = $2025 + 2 = 2027$

Type C: First Variable Incrementing

Example:

$a^m b^n$, $m \geq 1$, $n \geq 0$

Minimal string = "a"
States = $1 + 3 = 4$

Example:

$a^m b^n$, $m \geq 5$, $n \geq 0$

States = $5 + 3 = 8$

Type D: Both Variables Incrementing

Example:

$a^m b^n, m \geq 1, n \geq 1$

Minimal string = "ab"

Minimal length = 2

States = $2 + 2 = 4$

Example:

$a^m b^n, m \geq 5, n \geq 2025$

Minimal length = $5 + 2025 = 2030$

States = $2030 + 2 = 2032$

PART II: COMPOUND FINITE AUTOMATA

Compound automata combine two or more DFAs using Boolean logic.

1. Logical Operations

Operation	Meaning
AND	Accept only if both machines accept
OR	Accept if at least one machine accepts

2. Product (Grid) Construction

If:

Machine M_1 has m states

Machine M_2 has n states

Combined machine has:

$m \times n$ states

State Representation:

(q_i, q_j)

Transition Rule:

Each input symbol updates both components simultaneously.

Visualization Principle:

Symbol a $\rightarrow$ horizontal movement

Symbol b $\rightarrow$ vertical movement

3. Final State Determination

Operation	Final State Condition
Intersection (AND)	Both component states must be final
Union (OR)	At least one component state must be final

PART III: DIVISIBILITY AND MODULO COUNTERS

Languages involving divisibility use remainder-tracking states.

1. Single Divisibility Rule

If number of occurrences of a symbol must be divisible by n:

Required states = n

States represent remainders:

0, 1, 2, ..., n-1

2. Compound Divisibility

Total states = product of individual divisors.

Case	Constraint	Total States
Case 1	$a \bmod 3 \text{ AND } b \bmod 2$	$3 \times 2 = 6$
Case 2	$a \bmod 6 \text{ AND } b \bmod 8$	$6 \times 8 = 48$
Case 3	$0 \bmod 3 \text{ AND } 1 \bmod 5$	$3 \times 5 = 15$

3. Remainder Inequality Conditions

Example:

$$a \bmod 3 \geq b \bmod 2$$

Process:

Compute remainder of a-count modulo 3.

Compute remainder of b-count modulo 2.

Compare remainders.

Example:

String with 3 a's and 3 b's

$$3 \bmod 3 = 0$$

$$3 \bmod 2 = 1$$

Since $0 \geq 1$ is false, string is rejected.